

Software Factory facilitator guide

One guide for every session. Each session starts on its own page, so you can print one and hand it over. A session lists what to say, what to run, what the room does and what to ask, and it names the ideas it teaches. The attendee workbook explains each of those ideas once.

Read for	whoever is running a lesson, a module, the leadership workshop or office hours
Proven by	<code>make check</code> , <code>make prove</code> and <code>make e2e</code> in this repository
Printed by	<code>make book</code> , one part per session, each on a fresh page

Contents

01 Follow the card, not the label

Before any session

02 Design a Software Factory for Your Coding Agents

Lightning lesson, sixty minutes

03 Build Your First Software Factory Execution Harness

Lightning lesson, sixty minutes

04 Enforce Code Quality with Automated Agent Verifiers

Lightning lesson, sixty minutes

05 Automate Pull Requests with Background Coding Agents

Lightning lesson, sixty minutes

06 Design Your Software Factory Architecture

Cohort module one, two hours

07 Isolate Agent Work with an Execution Harness

Cohort module two, two hours

08 Route the Right Context to Every Task

Cohort module three, two hours

09 Make Agents Prove Their Work

Cohort module four, two hours

10 Build Self-Healing Engineering Loops

Cohort module five, two hours

11 Orchestrate and Ship Verified PRs

Cohort module six, two hours

12 Design an AI-Native Software Factory for Your Organization

Leadership workshop, one day

13 Office hours and capstone demo day

Weekly hour, plus the final session

Before any session

Follow the card, not the label

Note

The button that opens a session is not always called the same thing. A resting card offers Investigate, a card the factory is working offers Build, and Open session appears once a session exists and is idle. Whichever button is on the card is the one that opens what the run is doing. A guide that named a fixed button sent a presenter hunting for one that was not there.

Lightning lesson, sixty minutes

Design a Software Factory for Your Coding Agents

Architecture session. No live coding. The room leaves with a map they can hold against their own repository, and one named gap.

Ideas taught here: **six layers, boundary, context, skills, execution, verification, delivery, the growing prompt.**

Twenty minutes ahead, alone

Before anyone joins

Goal The demo is warm and the console has data in it. Nothing here happens while people watch.

Say Nothing. This segment is setup.

Run

```
$ make check
$ make score
$ make demo STEP=06
```

Check

- The score prints and every phase is green.
- The console is serving and lists finished runs with their timings.
- Your editor is on a clean branch, because the harness will create worktrees and you want that visible.

Ask *Nobody. Confirm the room can read a terminal at the back before you start.*

0:00 to 0:08

Naming the problem each team already has

Goal Everyone recognises the problem as theirs before any architecture appears.

Say

- Open with the shape of the day, not the technology. The function took seconds. Then came re-explaining the codebase, checking what else moved, and tidying the pull request.
- Name the wrong conclusion out loud: that the model needs to improve. Ask who has responded to a bad result by adding another paragraph to their instructions.
- Land the reframe: the generator is not the missing piece. There is nowhere safe for it to run and no reliable way to brief it. Nothing checks its work before it reaches you.

Run Nothing yet. Resist opening a terminal here.

Room sees Slides or a whiteboard only. Faces, not a screen.

Ask *How long is your instructions file now, and when did you last read all of it?*

0:08 to 0:28

The six jobs of a factory

Goal Attendees can name all six jobs and say which failure each one removes.

Say

- Walk the six in order, giving each one a single sentence and a symptom. Do not define them abstractly; define them by what goes wrong when they are missing.
- Spend the longest on the second and fifth, because those are where the room's pain actually is.
- Say the sentence that reframes the hour. These fail separately, so the fix depends on which one is missing.

Run

Show the map in the workbook and scroll through it.

Room sees

- The six-layer diagram, one heading at a time.
- The symptom under each, said aloud rather than written on the slide.

Ask *Pick the symptom you had this week. Which job was missing?*

0:28 to 0:40

Routing beats piling

Goal The room understands why their instructions file stopped helping, and what replaces it.

Say

- Describe the growth curve honestly. Each addition works, which is why it keeps happening, and accuracy still falls.
- Explain the competition for attention. The sentence that matters is now one of forty, and thirty-nine do not apply to this task.

- Show the alternative as a calculation rather than a philosophy. Files to be touched, owners of those files, rules for those owners, skills for that kind of work.

Run

```
$ make demo STEP=03
```

Room sees

- Two tasks, side by side, receiving visibly different material.
- The frontend task never being shown the money rules, and the reason why printed next to it.

Ask

What does a new engineer get told on day one that your agent is never told?

0:40 to 0:52

The full loop, run once

Goal

They see the finished thing move, so the layers stop being a diagram.

Say

- Narrate the stages as they happen and keep naming which job is doing the work.
- Stop on the refusal and give it a beat. A run declining to touch the money path is the most important thing on the screen, and it looks like nothing.
- Close the loop at the pull request and say plainly that this is where the machine stops.

Run

```
$ make demo STEP=06
```

Room sees

- An item claimed, a worktree created, checks running, a verdict, a draft pull request.
 - A second item stopping with a policy reason instead of a change.
 - The console updating with the stage timings for the run that just finished.
-

Ask

Which of those stages would you have let run unattended on your repository this morning?

0:52 to 1:00

Each attendee names one gap

Goal

Nobody leaves with a reading list. Everybody leaves with one named absence.

Say

- Ask them to write down which of the six they do not have, in their own words, before the call ends.
 - Say where the next sessions go and what each one builds, briefly, without selling.
-

Run

Leave the console on screen while taking questions.

Room sees

The repository address, and the workbook open at the overview.

Ask

Name the one you are missing. Type it in the chat so the room sees the spread.

0:35 to 0:45

The same policy, enforced by a third-party product

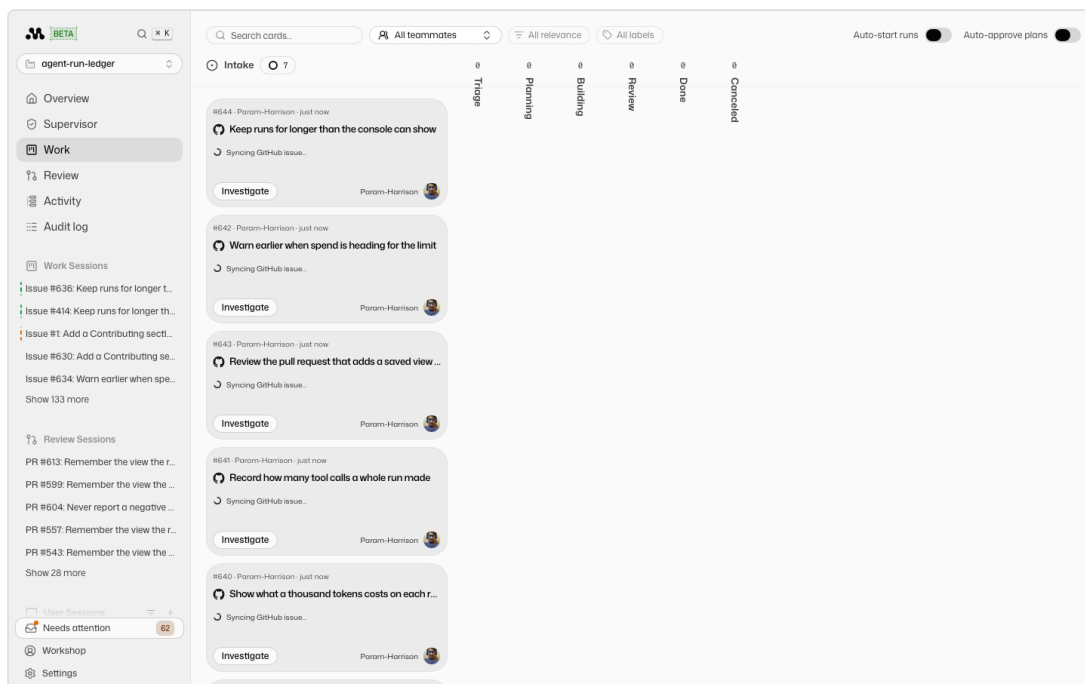
Goal They have seen the charter they just wrote being obeyed by software none of us built.

Say

- Point out that the board is reading the same ``.factory`` directory the hand-built loop reads.
- The value of a written policy is that a second implementation can honour it without being told anything else.
- Name what is missing on purpose: nothing has run yet, so nothing on screen is a claim about quality.

Run

```
$ make factory-up
$ make lab-reset
```



Every issue the repository declares arrives from one place. Nothing has started, and both switches are off.

Show on the board

Work to Intake to a card to Investigate

The charter is not read by magic

Mastra Factory has no knowledge of a .factory directory. What it reads is AGENTS.md, and the only reason these rules bind anything is that the file points at them and repeats the protected list. Delete it and every limit here becomes a comment.

Both switches are off, deliberately

Arrival and action are separate. An issue landing on Intake is not a run starting, and the gap between those two is where a person decides whether this is work at all.

Six cards, one repository

Nothing on this board knows what the ledger is. It knows a repository was connected and that its issues arrive here.

Room does

- Watches six cards appear without anybody filing them.
- Notices that Auto-start runs is off, so arrival and action are separate things.

Ask

If your policy file vanished, which of these six would a stranger get wrong first?

Lightning lesson, sixty minutes

Build Your First Software Factory Execution Harness

Hands on. Attendees follow along in their own repository and finish with an isolated place for agent work.

Ideas taught here: **worktree per task, the branch as claim, limits, executor interface.**

Thirty minutes ahead

Setup you do alone

Goal Nothing is left over from an earlier rehearsal, and only the main checkout is listed.

Say

- Nothing. Preparation only.

Run

```
$ make lab-reset  
$ git -C ../ledger worktree list
```

Check

- Only the main checkout is listed before you begin.
- A rehearsal leaves nothing behind, so the list is the same afterwards.

Ask *Nobody. Widen the terminal font now, because the room watches three runs and reads what they print.*

0:00 to 0:08

An agent that edited your checkout

Goal Everyone recognises an agent that edited a checkout they were working in, before any mechanism appears.

Say

- Say what you are working on before how. The ledger is a screen that shows what each agent run cost, where its time went and how it ended. A page asks it for zero rows and gets an error, not an empty list. The terminal will name it as issue 2.
- Put the comparison up. You hand it to an agent and carry on. By evening your files are half edited and your branch has moved.
- Show the six layers with the execution layer lit. Boundary, context and skills come before it builds. Execution, verification and delivery come while it builds and after.
- Say the session works alone and anyone can follow it. Then show the finished harness drawn whole.

Run

Slides only. Terminal open, nothing typed.

Room sees

- The comparison between what you filed and what you found by evening.
- One drawing of the six layers, with execution lit.
- One drawing of the finished harness, before any part of it.

Ask

Who has lost work to a background run? What did it cost to recover?

0:08 to 0:21

A worktree for every task

Goal Everyone has watched a task get its own worktree, a file written in it, and the main checkout stay clean.

Say

- Collect the war stories first, then draw the shared checkout and the branch that moved. Each story is a missing place, not a misbehaving tool.
- Run the demo for six minutes. Write a file in the new worktree, then show git status clean in the main checkout.
- Open `worktree.ts` beside the terminal: the name comes from the item, so the same item always maps to the same branch.
- Put the decision to the room: one shared checkout, or a worktree per task. Free and collides, against disk and an install and never.

Run

```
$ make demo STEP=02 create 2
$ git -C ../ledger worktree list
$ make demo STEP=02 remove 2
```

The implementation is `steps/02-execution/worktree.ts`. Ignore the make line that follows each run, it is make reading the argument as a target.

Room sees

- Branch `fq-2`, a path under `.worktrees`, and the workspace count going from 0 to 1.
- Two entries in the worktree list, then one again after the remove.

Ask

How many agent tasks would you run while you sleep, and where would each one install its dependencies?

0:21 to 0:34

Two runs on one item

Goal

They understand how ownership is settled without anything new to operate.

Say

- Pose the collision before showing the answer: two scheduled runs wake up, both see issue 2, both want to push a fix.

- Let someone propose a lock service, then price it: something to run, its uptime, and what happens when the holder dies.
- Run the demo for six minutes. The remote is a throwaway bare repository, so the race is real.
- Put the decision to the room: a lock service, or the branch you already have.

Run

```
$ make demo STEP=02 race 2
```

Read it in `steps/02-execution/claim.ts`: the push is never forced.

Room sees

- Both runs computing fq-2, run 1 continuing, run 2 stopping with the remote's own rejected line.
- A verdict saying exactly one run owns the work.

Ask

What else in your pipeline would be simpler if the coordination lived in a tool you already run?

0:34 to 0:47

Limits on what one task can reach

Goal

Attendees can state limits for one task in their own repository and know what happens when one is exceeded.

Say

- Separate the two questions. A worktree answers where the work happens. Sandbox limits answer what it can touch once it is there.
- Run the demo for six minutes. Read the profile table first, then the four refusals one at a time.
- Insist on the ending: exceeding a limit stops the run, and a warning in a log is not a limit.
- Put the decision to the room: limits the agent can edit, or limits it cannot.

Run

```
$ make demo STEP=02 limits
```

Declared in `steps/02-execution/limits.ts`, where a reviewer can read them.

Room sees

- A table of profiles, one per kind of task, and a display task writing inside its limits.
- A write to services/budget, a path out of the worktree, curl and the network each refused, with the reason.

Ask

List three commands you would never want a background run to execute. Why those three?

0:47 to 1:00

One issue, end to end

Goal

They can say what the harness gives, and what it does not, in one sentence each.

Say

- Walk the whole drawing: claimed, a worktree, sandbox limits on, an agent inside them, a fix on its own branch. The agent is the one part you can swap.
- Put the honest limit on screen. A worktree is not a sandbox. An allowed command can still be the wrong one, and nothing in this hour has read a line of the change.
- Say the offer once, then finish on the closing question.

Run

Leave the worktree list and a clean status side by side on screen.

Room sees

- The repository address and the three files worth reading first.
- The closing question of the deck.

Ask

Where would your checkout not work? Type it in the chat.

0:35 to 0:45

The same rule in a third-party sandbox

Goal

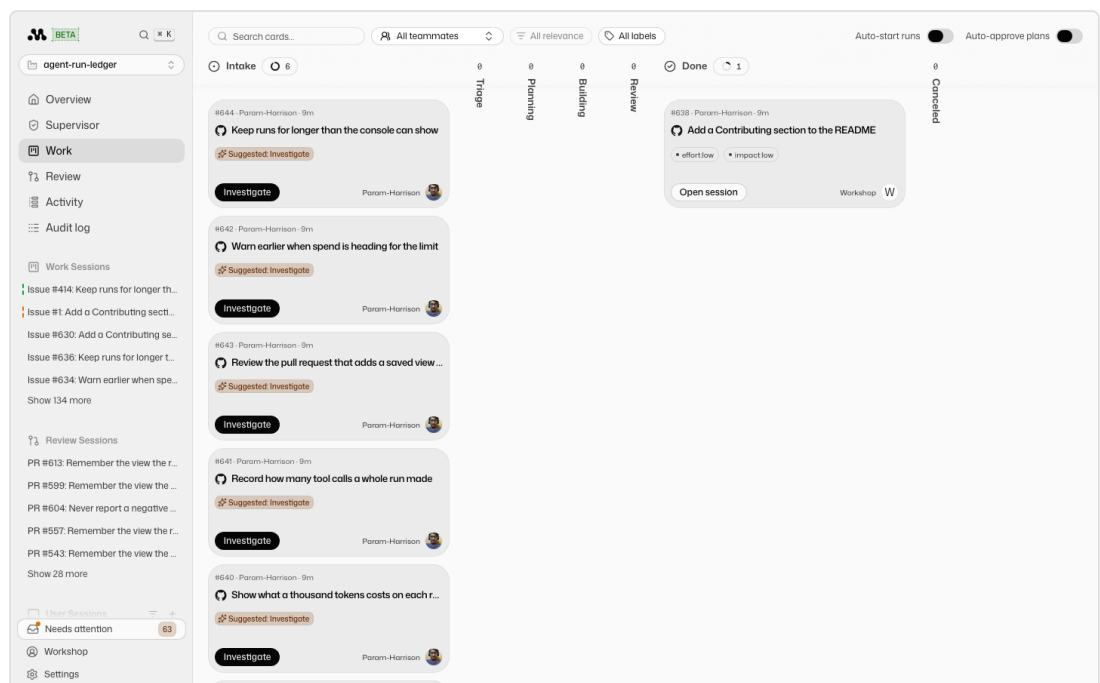
They can say why an isolated checkout is a property of the harness rather than a courtesy.

Say

- Two harnesses now: the one built in this lesson and the one inside the product.
- Both give a run its own tree, for the same reason. In a shared tree you cannot tell two runs apart when one breaks.
- Show where the checkout lives, and that deleting it costs nothing.

Run

```
$ ls ../mastra/.sandboxes || true
```



A run holds a checkout of its own. The repository on your disk is untouched while it works.

Show on the board

Work to a card in Building to its session to the file tree

A checkout of its own, every time

The session holds a clone under .sandboxes. Delete it while the run is going and the run dies; delete your own working tree and nothing happens, which is the whole point of the boundary.

The one that cost me an evening

A review once checked a pull request out in the Factory server's own directory rather than a sandbox. The files docker-compose.yml and package.json vanished, and the only symptom was "compose file is invalid". Ask the room where they would have looked first.

Sandboxes are cattle

They are named by run, not by branch, and nothing in them is worth keeping once the pull request exists.

Room does

- Opens the sandbox directory and confirms it is a real clone.
- Confirms their own working tree has no new branch.

Ask

What breaks first if two runs share one checkout: the build, the test, or the diff?

Lightning lesson, sixty minutes

Enforce Code Quality with Automated Agent Verifiers

The session that changes how people review. Most of the hour is spent on one idea: undo the change and watch the test go red.

Ideas taught here: **the verdict line**, **holdout tests**, **the cold reader**, **the negative proof**, **Doer and Tester**.

Thirty minutes ahead

Setup you do alone

Goal Every command has run once, so nothing is compiling on stage.

Say

- Nothing. Preparation only.

Run

```
$ make lab-reset
$ make demo STEP=04 gate
$ make demo STEP=04 negative-proof
$ make demo STEP=04 verify
$ make demo STEP=04 holdout
```

Check

- The gate ends in one verdict line and nothing summarises it afterwards.
- verify names the two findings on the saved-view change, with its reason.

Ask *Nobody. Have fixtures/saved-view.ts open in a second window, ready to show.*

0:00 to 0:08

A green pull request that was still wrong

Goal Everyone recognises a green pull request that was still wrong, before any mechanism appears.

Say

- Say what you are working on before how. The ledger is a screen that shows what each agent run cost, where its time went and how it ended. Someone asked for a small feature: remember the filter a reader has set. The terminal will name it as issue 6.
- Put the comparison up: the agent wrote it and its tests, the suite was green and you approved. A month later it stores more than was asked and an old test had been weakened.
- Show the six layers with the verification layer lit. Boundary, context and skills come before it builds. Execution, verification and delivery come while it builds and after.
- Say the session works alone and anyone can follow it. Then show the finished verifier drawn whole.

Run Slides only. No commands yet.

Room sees

- The comparison between green and approved, and a month later.
- One drawing of the six layers, with verification lit.
- One drawing of the finished verifier: a gate, tests proven to bite, a cold reader, one verdict.

Ask *Who has approved a green pull request that was wrong?*

0:08 to 0:21

A missing check must be louder than a failing one

Goal Attendees can state why a missing check must be louder than a failing one.

Say

- Draw green as a claim, not a proof: the tests pass, and the writer chose the tests.
- Run the demo for six minutes. Read the verdict line aloud, then the note under it. A summary of the line is a claim. The line is evidence.
- Run it again with the required check removed. It does not pass and it does not fail, it says misconfigured.
- Put the decision to the room: pass or fail, or a third state for nothing ran.

Run

```
$ make demo STEP=04 gate
```

The three outcomes are in `steps/04-verification/gate.ts`.

Room sees

- VERDICT: PASS after the repository's own test command ran.
- The same gate with the check removed ending in MISCONFIGURED, with no test command run.

Ask *If your pipeline lost its test step tonight, how long before anyone noticed?*

0:21 to 0:34

Revert the change and watch the test fail

Goal Everyone leaves able to apply this to code that has no agents anywhere near it.

- Say**
- Show the pull request replacing an assertion in apps/console with one that passes for any input. The suite is green because the test was changed.
 - Run the demo for six minutes. Green with the change, red with it undone and the test kept, green again restored.
 - Then run it under a test that checks nothing and let the FAIL sit on screen. Same suite, a minute apart.
 - Put the decision to the room: undo every test, or a sample.
-

Run

```
$ make demo STEP=04 negative-proof
$ make demo STEP=04 negative-proof --toothless
```

The revert, rerun and restore are in `steps/04-verification/verifier.ts`.

- Room sees**
- The change reverted, the test going red, the change restored.
 - The second run failing: the suite passes and it proves nothing.
-

Ask *Take the last test you wrote. Would it have gone red without your change?*

0:34 to 0:47

A cold reader reviews the change

Goal They understand why the reviewer is deliberately denied the story of the work, and why some checks are hidden from the writer.

Say

- Tell the anchoring story from human review, which everyone recognises, then apply it to a second reader.
- Run verify for six minutes. Show what the cold reader is given, the issue and the diff, then what it is not given, then its findings.
- Read the two findings aloud. The change also stores the sort order, column widths and last run opened. And a weakened assertion is why the suite is green.
- Run holdout: five checks that live outside the target, none of them visible to the writer.
- Put the decision to the room: the writer's own tests, or a suite it cannot read.

Run

```
$ make demo STEP=04 verify  
$ make demo STEP=04 holdout
```

The diff the cold reader sees is `fixtures/saved-view.ts`.

Room sees

- A verdict of fail on a change whose own suite was green, with both findings quoted.
- Five independent checks, each saying what it protects.

Ask *Which of your services would you write five holdout tests for first?*

0:47 to 1:00

Pull request to verdict

Goal They can say what a verdict proves, and what it cannot judge.

Say

- Walk the whole drawing. Green on its own tests, then the gate's line, each test undone and the holdout tests. A cold reader follows, and the verdict is FAIL with both reasons quoted.
- Put the honest limit on screen. A gate cannot judge whether the design is right. A cold reader will sometimes reject good work. A holdout suite starts ageing the day it is written.
- Say the offer once, then finish on the closing question.

Run

Leave the verdict on screen while taking questions.

Room sees

- The verdict line with both findings beneath it.
- The closing question of the deck.

Ask *Would your last test have gone red without your change? Type it in the chat.*

0:35 to 0:45

A reviewer that can reject

Goal They accept that a reviewer which never rejects anything has not been tested.

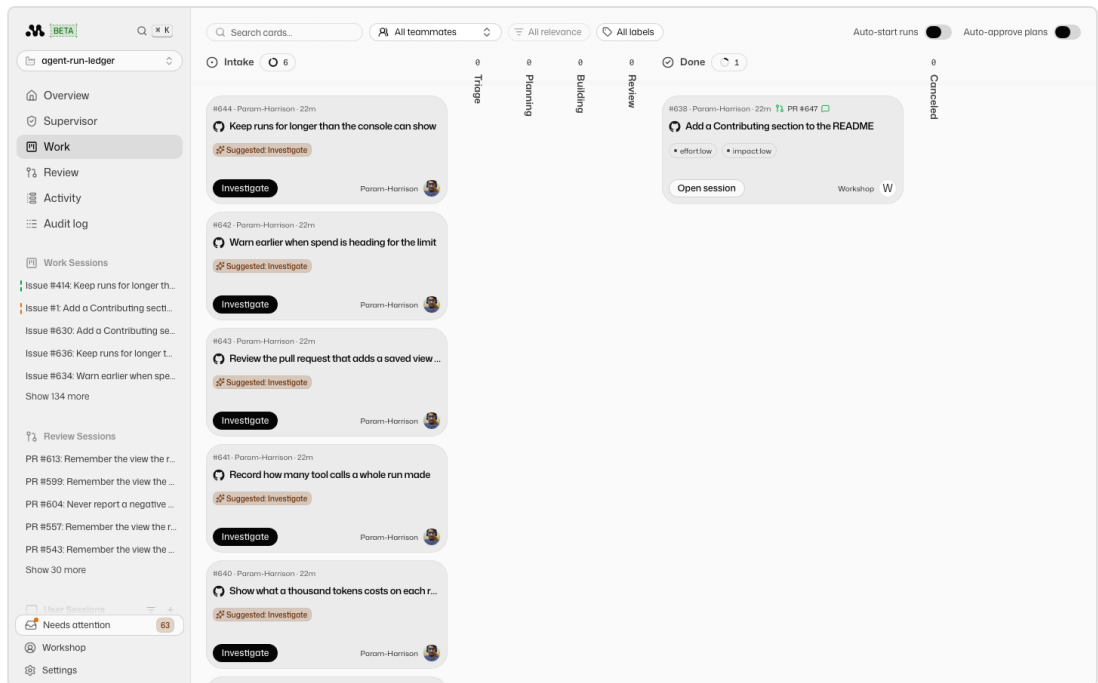
Say

- The interesting artefact is the refusal, not the approval.
- Read the two findings aloud and note that one of them is a weakened assertion, which no path rule could have caught.

- This is why the charter forbids editing a test to turn a suite green. It is a property of the change, not of a filename.

Run

```
$ make lab-reset
```



The verdict line, with the two findings that produced it.

Show on the board

Review to the pull request to Conversation to the verdict line

The verdict is the first line, on purpose

A reviewer that buries its conclusion in prose cannot be checked by anything. One line, one of two values, and everything after it is the argument for it.

A weakened assertion is not a path rule

Changing toBeCloseTo(1, 10) to toBeGreaterThanOrEqual(0) touches no protected file. No glob catches it, which is why the charter forbids it in words and the reviewer has to be the thing that notices.

A picture is the one artefact a test cannot read

Four screenshots in this repository were a loading spinner, and every assertion around them passed. Show the room the check that refuses to photograph a page still saying it is loading.

Room does

- Reads the verdict line and both findings.
 - Identifies which finding a glob could have prevented and which it could not.
-

Ask

Your suite went green after an edit to a test. What would tell you which kind of edit it was?

Lightning lesson, sixty minutes

Automate Pull Requests with Background Coding Agents

A live run from an untouched issue to a reviewable pull request. The demonstration is the session; commentary fits around it.

Ideas taught here: **the queue**, **a request with its evidence**, **the merge gate**, **the run record**.

Thirty minutes ahead

Reset the board

Goal Every command has run once, so nothing is compiling on stage, and the board is reset.

Say

- Nothing. Preparation only.

Run

```
$ make lab-reset  
$ make demo STEP=06 triage  
$ make demo STEP=06 run 5  
$ make demo STEP=06 refused  
$ make demo STEP=06 try-merge
```

Check

- triage prints a table with a column for what may be done and a column for why.
- run 5 ends in refused, with zero files changed.

Ask *Nobody. Decide now what you will do if the network drops, and keep the recording as the fallback.*

0:00 to 0:08

The issue nobody owns

Goal Everyone recognises the issue nobody owns, before any mechanism appears.

Say

- Say what you are working on before how. The ledger is a screen that shows what each agent run cost, where its time went and how it ended. One total on it can be wrong, and the fix touches four languages. The terminal will name it as issue 4.
- Put the comparison up: filed, and three weeks later still open, with the console adding the numbers itself.
- Show the six layers with the delivery layer lit. Boundary, context and skills come before it builds. Execution, verification and delivery come while it builds and after.
- Say the session works alone and anyone can follow it. Then show the finished pipeline drawn whole.

Run Slides only. Resist opening a terminal here.

Room sees

- The comparison between an issue filed and an issue forgotten.
- One drawing of the six layers, with delivery lit.
- One drawing of the finished pipeline, ending at a person.

Ask *What is the oldest cross-team issue in your tracker?*

0:08 to 0:21

Triage before anything runs

Goal The room can say why every item is sorted, with a reason beside it, before an agent starts.

Say

- Draw issue 4: it sounds like one field and it touches the schema, the Go service, the TypeScript package and the console.
- Run the demo for six minutes. Read the table: what each item may do, and why. Issue 2 builds, issue 4 is a proposal naming three targets, issue 5 is refused.
- Read the two lines under the table. Triage touched no file.
- Put the decision to the room: a queue service, or the labels and branches you already have.

Run

```
$ make demo STEP=06 triage
```

Ignore the make line that follows, it is make reading the argument as a target.

Room sees

- A table with a column for what it may do and a column for why.
- Issue 5 refused, and the reason is the protected path services/budget.

Ask *What would you want a queue to do that your issue tracker cannot?*

0:21 to 0:34

Refused before a file is touched

Goal They see a refusal that writes nothing, with the rule quoted, and can say why that is the successful outcome.

Say

- Set it up: issue 5 asks for an earlier spend warning in the Rust budget service. The change is one number, and that number decides when the money stops.
- Run issue 5 for six minutes. Ask the room what they expected, then point at the zero.
- Run refused to show the rule and its reason in plain words.
- Put the decision to the room: refuse outright, or propose a plan to a person.

Run

```
$ make demo STEP=06 run 5
$ make demo STEP=06 refused
```

Room sees

- The verdict quoted, REFUSED protected: services/budget, and a last line of zero files changed and zero cost.
- The README item refused because the factory's own configuration is off limits.

Ask

Which of your paths would you refuse outright, and which would you only ask a person about?

0:34 to 0:47

The draft a reviewer opens

Goal

They can say what is in the draft a reviewer opens, and why nothing merges without a named person.

Say

- Show the draft: every file that changed, every verdict quoted, time and cost, and what a person still decides. Open record.ts and show the function that writes the body.
- Say plainly that a run to a draft needs a recorded or live agent. So the draft is on the slide and in the code, and is not run here.

- Run try-merge for six minutes: a merge attempted, the hook catching it, and the refusal.
- Say the hook is a guardrail, and a guardrail an agent can edit is not a boundary. The enforcement that counts is branch protection on the remote.
- Put the decision to the room: every run is a draft, or a low tier merges itself.

Run

```
$ make demo STEP=06 try-merge
```

The draft body is written in `steps/06-delivery/record.ts`.

Room sees

- A merge attempted and refused: nothing merges without a named engineer, on any tier.
- The function that writes the draft body, with every verdict quoted rather than summarised.

Ask

What is the smallest change you would let merge with nobody looking?

0:47 to 1:00

Issue to draft

Goal

They can say where the ceiling is, and it is not the number of agents.

Say

- Walk the whole drawing. The review queue has room, then the boundary decides. Issue 5 stops. Issue 4 is proposed, and once a person accepts the plan it is built, checked and opened as a draft.
- Put the honest limit on screen. Nothing here judges whether a plan is right. More agents cannot cut a wait that is a person reading.
- Say the offer once, then finish on the closing question. Give the take-home last: the smallest factory is one signal, one agent and one gate.

Run

Leave the pull request list on screen while taking questions.

Room sees

- The whole run as one drawing, ending at a named person.
- The closing question of the deck.

Ask

How many pending decisions can you absorb in a week? Type it in the chat.

0:35 to 0:45

The pull request as the interface

Goal

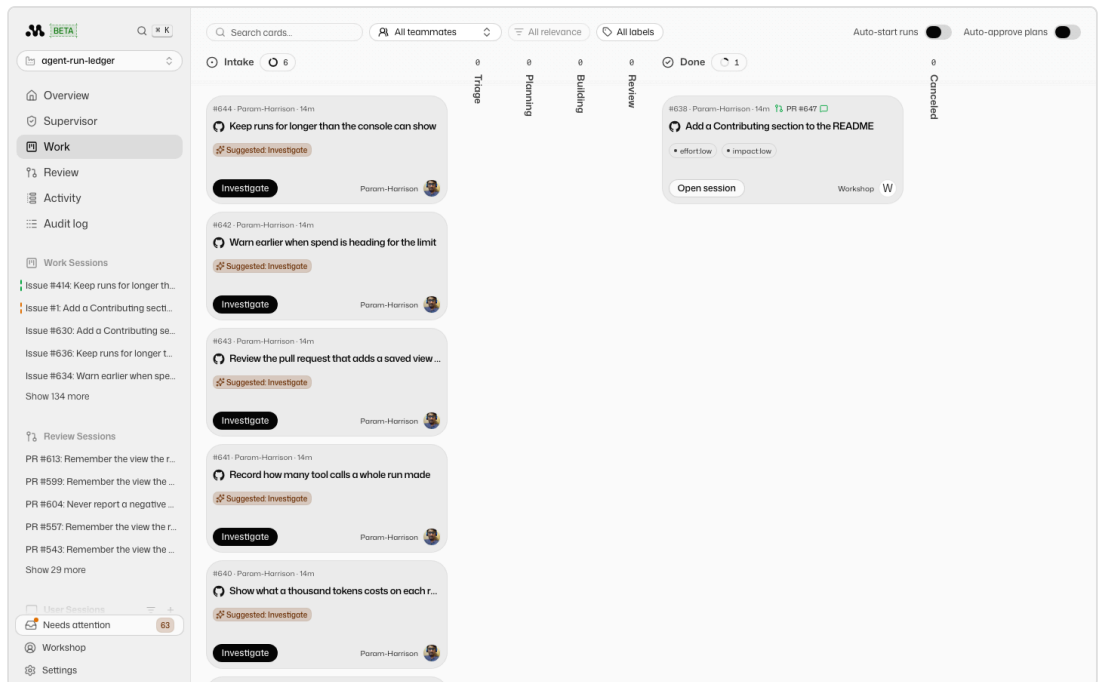
They can describe the hand-off point between a machine and a person in one sentence.

Say

- Every route in this material ends at a draft, on every tier, without exception.
- The pull request is where the evidence lives, which is what makes it reviewable by someone who watched none of the run.
- Contrast that with a chat transcript, which nobody can audit six weeks later.

Run

```
$ gh pr list --repo learnwithparam/agent-run-ledger
```



The change, its evidence, and nothing merged.

Show on the board

Review to the pull request to Files changed to the plan artefact

The evidence travels with the change

The plan the run wrote is committed beside the diff, so a reviewer who watched none of it can see what was intended before reading what was done.

Nothing merges itself, on any tier

Every route in this material ends at an open pull request. The charter says so and the four shapes do not argue about it.

A decision is not the work

The board records a decision succeeding within seconds while the agent goes on working for minutes. Anyone timing a stage from the decision record will report a factory four times faster than it is.

Room does

- Opens the pull request and finds the commands the run actually executed.
- Confirms nothing was merged by anything except a person.

Ask

What in this pull request would you need if you were reviewing it a month from now?

Cohort module one, two hours

Design Your Software Factory Architecture

Everyone arrives with a repository they are allowed to change. They leave with a written policy for it that a run can read, and that they could defend in a design review.

Ideas taught here: **the charter**, **autonomy by consequence**, **protected paths**, **back-pressure**, **the ownership graph**.

0:00 to 0:20

Applying it to each attendee's repository

Goal Each person has named the area of their codebase where an error would be most expensive.

Say

- Set the rule in the first minute. Every exercise happens on their code. The lab only shows a worked example.
- Ask them to write down the single directory they would least like a machine to edit, and why it earns that.
- Collect three answers aloud. The reasons differ, and hearing that is what stops the policy becoming a template they copy.

Run

```
$ git clone <their repository> && cd <repository>
$ ls
```

Room does

- Opens their own project beside the lab.

- Writes one directory name and one sentence of justification in the shared document.

Ask

Would a small edit there be safer than a large one somewhere else? Say why out loud.

0:20 to 0:45

Autonomy tiers set by recoverability

Goal

They can defend a tier for their repository using recoverability rather than size.

Say

- Offer the two candidate axes and let the cohort argue for a few minutes before you settle it.
- Use the comparison that ends the argument. Set a huge rewrite on a project nobody uses against a tiny edit to a payment path.
- Then complicate it usefully. One tier for a whole repository is too blunt once areas differ, which is why the next exercise is per area.

Run

```
$ cat $REPO/.factory/charter.md
```

The rules live in the repository they govern. The code that reads them is `steps/01-boundary/policy.ts`, and it names no project.

Room does

- Drafts a tier line for their own repository.
- Argues one case with a neighbour, which surfaces the disagreements early.

Ask

What is the worst thing an unattended run could do to you, and how long would undoing it take?

0:45 to 1:20

Who owns which paths

Goal Everyone has a graph where every directory has exactly one owner, and the build refuses when one does not.

Say

- Introduce the graph as the thing that makes policy specific rather than vague. Areas, the paths they own, and the commands that check them.
- Build the lab's version live, then break it by adding an unclaimed directory so the room sees the refusal.
- Insist on the property that matters. Guessing an owner is worse than failing, because a wrong guess is silent.

Run

```
$ node steps/01-boundary/demo.ts graph
$ mkdir -p target/services/orphan && touch
target/services/orphan/main.go
$ node steps/01-boundary/demo.ts graph
```

Room does

- Sees every area listed with its paths and its checks.
- Sees the second run refuse and name the directory nobody claimed.
- Writes the first version of their own graph.

Ask *Which directory in your project was hardest to assign, and what does that tell you?*

1:20 to 1:45

The limit on pending decisions

Goal Each person has committed to a limit on pending decisions, chosen from their own week.

Say

- Ask how many changes they genuinely absorb in a week, then ask them to halve it for anything they did not write.
- Explain what the limit protects: not the machine, the person. When it is reached, starting more work is the wrong move.
- Show the refusal in the lab so the number is visibly load bearing rather than advisory.

Run

```
$ node steps/01-boundary/demo.ts capacity
```

Room does

- Watches a run decline to start because the review queue is full.
- Puts a number in their own charter and says it aloud.

Ask

What happens today when that number is exceeded, and who notices first?

1:45 to 2:00

Homework: a committed policy file

Goal

Everyone has a committed policy file in their own repository before the next session.

Say

- Set the deliverable precisely: committed, not drafted, because the next session reads it.
- Warn about the common mistake, which is writing something so cautious that nothing can ever run.

Run

```
$ node steps/01-boundary/demo.ts lint-charter <path to theirs>
```

Room does Runs the check against their own draft and fixes what it names.

Ask *Bring the line you are least sure about to Friday.*

1:35 to 1:50

Watching a limit hold

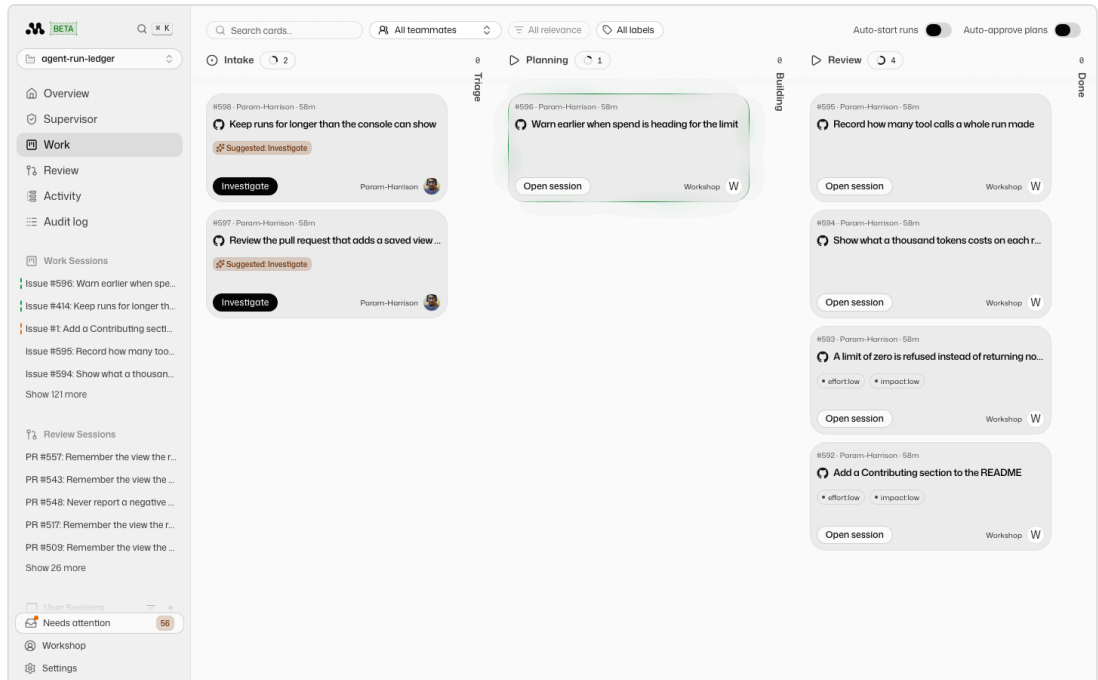
Goal Each person has seen their own class of protected path refused rather than described.

Say

- A limit that has never stopped anything is a sentence, not a limit.
- Walk the evidence backwards: no branch on the remote, no pull request, and a written reason that names the path.
- Then the uncomfortable part. Ask whether the refusal came from the charter or from the wording of the issue, because only one of those generalises.

Run

```
$ make profile NAME=enterprise  
$ make lab-reset
```



The item reaches planning and stops. No branch, no pull request, no edit.

Show on the board

Work to the money item to Planning to its session

Read the reason, not just the refusal

A run that stops because the issue said "a person decides this one" looks identical to one that stops because the charter protects the path. Only the second generalises to their repository, and the difference is a sentence in the record.

Nothing was written

Check the remote for a branch, then for a pull request. A refusal that leaves either behind is not a refusal, and this is the check to run on their own factory.

Protected and refuse are different stops

A refused target stops after planning with a plan nobody may act on. A protected path stops before planning starts. Enterprise uses both and scaleup uses one.

Room does

- Applies a shape to their own repository and names one path it would refuse.
- Checks the remote afterwards and confirms nothing was created.

Ask

Which of your protected paths has ever actually stopped a change? If none, is it protecting anything?

Cohort module two, two hours

Isolate Agent Work with an Execution Harness

The build session. By the end an agent can change code in their repository while their own branch stays exactly as they left it.

Ideas taught here: **the worktree**, **claiming by branch**, **declared limits**, **the executor**.

0:00 to 0:15

Reading each other's policies

Goal The homework gets read by someone other than its author, which is where the vague lines get found.

Say

- Pair them and give a single instruction: find one line you could not act on without asking the author a question.
- Bring two examples back to the whole group and rewrite them live.
- Make the standard explicit. A policy is finished when a stranger could apply it the same way you would.

Run Nothing. Documents on screen, in pairs.

Room does Edits at least one line based on somebody else's confusion.

Ask *Which line did your partner read differently from how you meant it?*

0:15 to 1:00

Building it together

Goal Everyone has a working create and remove cycle in their own project, written by them.

- Say**
- Write it in front of them from an empty file rather than opening the finished version. The cohort types along.
 - Stop at each decision and name the alternative. A clone instead of a worktree. A random name instead of a derived one. A global cleanup instead of a per-task one.
 - Have everyone prove the same property on their own repository before moving on. A harness nobody has proven is one nobody trusts.
-

Run

```
$ node steps/02-execution/demo.ts create 7
$ git -C .worktrees/fq-7 status
$ git status
$ node steps/02-execution/demo.ts remove 7
```

- Room does**
- Creates a workspace, writes into it, and confirms the main checkout is untouched.
 - Removes it and confirms nothing is left behind.
-

Ask *How long did the first one take on your project, and what dominated that time?*

1:00 to 1:35

Reach, declared per task

Goal Each person has limits their own reviewer could read, and has watched one stop a run.

Say

- Insist on the placement before the content. Limits go where a reviewer will see them, not inside instructions to the agent.
- Work through a task that legitimately needs more reach, so the rule is not simply deny everything.
- Show the enforcement path and be honest about where it is advisory and where it is real.

Run

```
$ node steps/02-execution/demo.ts limits
$ node steps/02-execution/demo.ts limits --violate
```

Room does

- Watches a refused write name the path and the rule behind it.
- Writes limits for one real task in their own project.

Ask *Which of your tasks genuinely needs the network, and how would you keep that from becoming the default?*

1:35 to 2:00

The assignment for the week

Goal Everyone knows exactly what to produce and what will be checked on Monday.

Say

- State the deliverable as something observable: a task that runs start to finish in isolation on their repository.

- Predict the two failures they will hit, so neither costs an evening. Dependencies are not installed in the new workspace. Generated files are ignored by the clone.

Run

```
$ node steps/02-execution/demo.ts selftest
```

Room does

Runs the self-check against their own harness and reads what it reports.

Ask

Post your isolation time in the channel when you have it running. We compare on Monday.

1:35 to 1:50

A queue limit is about attention, not capacity

Goal

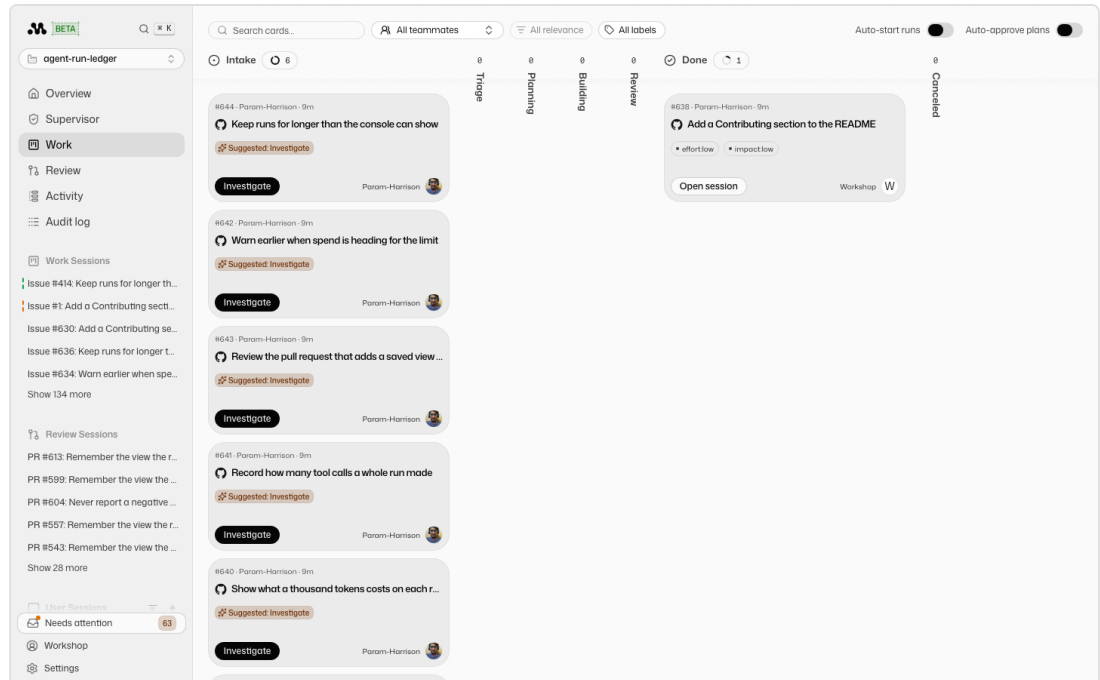
They can explain why a queue limit is about attention rather than machine capacity.

Say

- Generation is cheap and review is not, so the constraint worth declaring is how many things may be waiting on a person.
- Show the cap in the charter beside the board and let them find the mismatch if there is one.
- A factory producing faster than a team can read is not more productive, it is further behind.

Run

```
$ cat $REPO/.factory/charter.md
```



Several items in flight at once, each in a different phase.

Show on the board

Work to the board header to the phase counts to Settings

Work items come from events, never from polling

The reconcile sweep patches and closes items that exist; its counts have no created. The board fills because a webhook arrived, which is why a lab GitHub cannot reach delivers those events to itself.

The cap is about reading, not generating

Compare the number waiting with the stop condition in the charter. Generation is cheap; the constraint worth declaring is how many things may be queued against one person's attention.

Two boards, not one

Issues live on Work and pull requests live on Review, with their own phases. Teams that conflate them report a week of finished work that nobody has read.

Room does

- Finds the stop condition in the charter.
- Counts what is waiting on the board and compares the two numbers.

Ask

At what number of open pull requests does your team stop reading them properly?

Cohort module three, two hours

Route the Right Context to Every Task

The session where the instructions file gets taken apart. Everyone leaves with a router they can test, and two skills lifted out of prompts they were rewriting by hand.

Ideas taught here: **the router**, **writing a skill**, **why prompts grow**, **affected areas**.

0:00 to 0:20

Sorting your instructions into three piles

Goal Everyone has sorted their own instructions into three piles and seen how little of it any one task needs.

Say

- Give them three piles and let them sort. Facts about this repository. Ways of working. Things that apply to one task only.
- Ask for a show of hands on how much landed in the third pile. It is usually most of it, and seeing that is the argument.
- Name what each pile becomes before moving on, so the rest of the session has somewhere to put things.

Run

```
$ wc -l <their instructions file>
```

Room does Annotates their own file with three markers and reports the split.

Ask *Which line in there was added after one bad result and never revisited?*

0:20 to 1:05

Assembling context as a function

Goal Each person has a function that turns a task into its material, and a test that pins what it returned.

Say

- Derive the inputs with the room rather than presenting them. What does the task say it will touch, who owns those paths, what rules apply there.
- Build the lab's router live and keep it small. If it needs a diagram it has become a framework.
- Write the test immediately after, because the point of code over prose is that it can be asserted. Give the router a task and pin what came back.

Run

```
$ node steps/03-context/demo.ts route 12
$ node steps/03-context/demo.ts route 31
```

The router is `steps/03-context/router.ts`.

Room does

- Sees two tasks receive visibly different material, with the reason for each inclusion.
- Writes a first router against their own ownership graph.

Ask *What did your router include that the task did not need? Remove it and say what broke.*

1:05 to 1:40

Moving a procedure from a prompt into a skill

Goal Everyone has written one skill that states an outcome and its evidence, without scripting the keystrokes.

- Say**
- Ask what they have retyped more than three times this month. That is the first skill, and they already know what it is.
 - Show the failure before the good version. A long procedure is worse than the agent's own judgement, and it goes stale within a month.
 - Give the three parts and hold them to it. The outcome, the constraints that are not obvious from the code, and what proves it is done.

Run

```
$ ls steps/03-context/skills
$ cat steps/03-context/skills/change-the-shared-contract.md
```

- Room does**
- Reads one worked example, then closes it and writes their own.
 - Swaps with a neighbour to check the outcome is stated clearly enough to act on.

Ask *Could this skill be copied to a different repository unchanged? If not, some of it is context.*

1:40 to 2:00

Why mapping files to areas ships a break

Goal They understand why a naive mapping of files to areas ships a break, and have the dependency edge that fixes it.

- Say**
- Pose it as a puzzle. One file changed, one area owns it, one suite ran, everything green, production broken. What happened?
 - Draw out the edge: three areas are built against that definition, so all three are affected.
 - Note the second use of the same graph. It also tells the router which rules a task needs, so this is one calculation serving two purposes.
-

Run

```
$ node target/tools/affected.mjs --since HEAD~1
```

Room does Sees one edited definition select three areas and three different check commands.

Ask *Where is your shared definition, and how many places restate it today?*

1:35 to 1:50

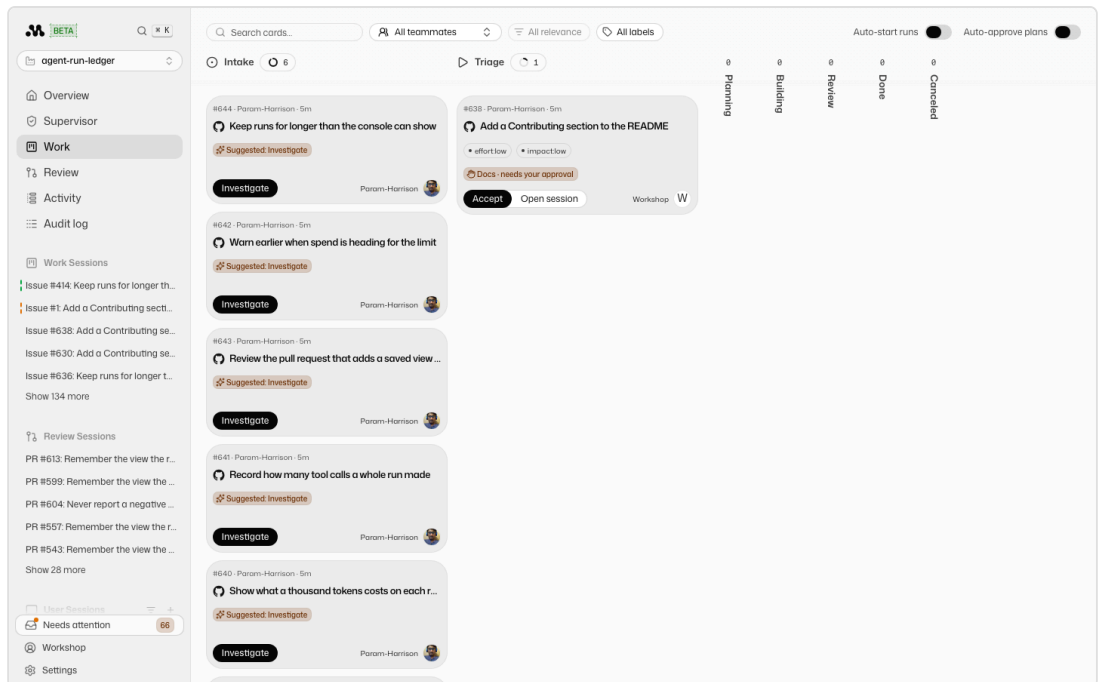
What the assessment was grounded in

Goal They can tell an assessment grounded in the repository from one grounded in the issue text.

- Say**
- Read the file paths in the assessment and check every one exists.
 - An assessment that names line numbers has opened the file; one that paraphrases the issue has not.
 - This is the cheapest quality signal available before any code is written, and it costs one read.
-

Run

```
$ make status
```



The assessment, with the files it names and the confidence it claims.

Show on the board

Work to a triaged card to its session to the assessment table

Every path it names, check one

An assessment that cites a file and a line number has opened the file. One that restates the issue in different words has not, and the two read alike until somebody checks.

The best outcome is sometimes no change

One run here answered a request for a cost column by proving the column already existed. It showed line numbers and five real currency values read from a live service. The issue was wrong and the run was right.

Confidence is a claim, not a feeling

The table states high or low confidence. Ask what evidence would have to be missing for that word to be wrong, and whether the run went and looked for it.

Room does

- Verifies two cited paths against the repository.
- Marks the assessment as grounded or not, and says which evidence decided it.

Ask

Which sentence in this assessment could have been written without opening the repository?

Cohort module four, two hours

Make Agents Prove Their Work

The longest build of the course. Everyone finishes with checks that fail closed and a reviewer that reads their diffs without being told the story.

Ideas taught here: **the result line, held-out checks, an independent read, proving the test bites, evidence rules, one definition.**

0:00 to 0:30

One script, one result line

Goal Everyone's checks are behind one command that says one thing, and a missing check is louder than a failing one.

Say

- Start from where they are. Most have commands scattered across a pipeline file, a readme and somebody's shell history.
- Gather them into one script live. Then add the ending that makes it useful: one exact line that nothing may restate.
- Do the deletion demonstration on their own project, not the lab. Remove a required command and show that the result is neither green nor a failure.

Run

```
$ bash steps/04-verification/gates.sh
$ bash steps/04-verification/gates.sh --without-tests
$ echo $?
```

Room does

- Writes one script for their own repository with the same ending.
- Confirms three distinct exit codes for passing, failing and misconfigured.

Ask *Which of your checks is currently optional, and who decided that?*

0:30 to 1:00

Holdout tests the writer never sees

Goal Each person has a small set of behavioural checks kept out of the implementing agent's reach.

Say

- Explain the circularity plainly: when the same run writes the tests and the code, green means the code agrees with itself.
- Choose what to hold out together. The contract and the behaviour a user would notice, not the internals.
- Head off the objection before it arrives. This is a sample, not a second suite, and a handful is enough.

Run

```
$ ls steps/04-verification/holdout
$ node steps/04-verification/demo.ts holdout
```

Room does

- Sees the holdout set run after the change, against work that never saw it.
- Picks three behaviours in their own project worth holding out.

Ask *What behaviour would embarrass you most if it broke quietly? Start there.*

1:00 to 1:30

Proving a trusted test can fail

Goal Everyone has run it against a test they already trusted, and some of them are surprised.

Say

- Have them pick a test they wrote themselves and are confident in. The confidence is the point.
- Walk the sequence together: undo the change it covers, run it, expect red, restore.
- Ask who found one that stayed green. There is usually at least one, and that discovery does more teaching than the explanation.

Run

```
$ node steps/04-verification/demo.ts negative-proof  
$ node steps/04-verification/demo.ts negative-proof --toothless
```

Room does

- Applies the sequence to one of their own tests by hand.
- Wires it into their loop so it happens on every change rather than when remembered.

Ask *How many of your tests have never once been red? What does that number mean?*

1:30 to 2:00

Deciding what a result may claim

Goal Their pipeline downgrades a conclusion whose evidence is missing, without consulting what the agent said.

Say

- Separate the two jobs sharply. Investigation is where judgement helps. The sentence a person will act on gets no freedom at all.
- Write the rules as ordinary code in front of them, reading the artifacts the run left behind.
- Connect it to the cross-language case. A definition changes in one place. The proof is three suites agreeing, not one claim that it went fine.

Run

```
$ node steps/04-verification/demo.ts evidence
$ node steps/04-verification/demo.ts evidence --lost-artifact
```

Room does

- Sees a confident conclusion downgraded because its proof could not be found.
- Writes one such rule for their own repository.

Ask

What does your pipeline currently believe purely because something reported it?

1:35 to 1:50

A gate failing in one language

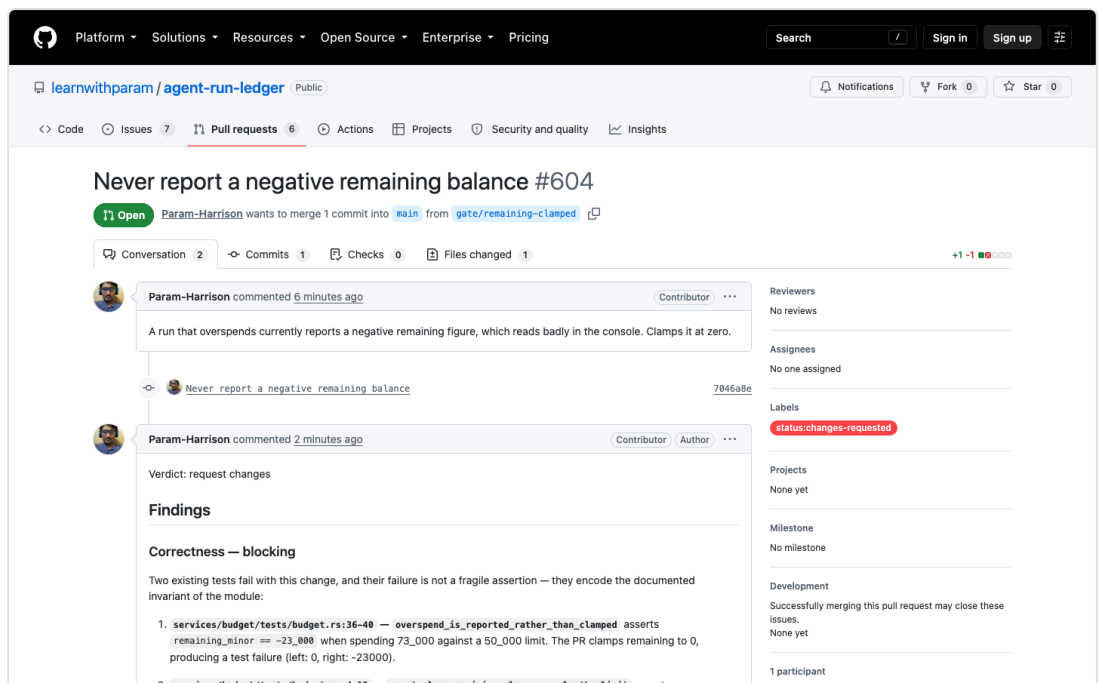
Goal

They can defend running only the checks a change reaches, and name what that misses.

Say

- The ownership graph decides which suites a change is measured by, so a schema edit is judged by all three.
- Show the failure and note that the language it failed in is not the language the issue named.
- Then name the cost honestly: a wrong graph means a real failure nobody ran.

Run

\$ **make** check

One change, three suites, and only one of them red.

Show on the board

Work to the cross-stack item to its session to three suites, one change

Only the suites a change reaches

The ownership graph decides what runs. A schema edit is judged by all three languages; a console change is judged by one. Show which ran and which did not.

A wrong graph is a failure nobody sees

If a directory is claimed by the wrong target, its suite never runs against changes that reach it. That is the cost of this design and it belongs in the room rather than in a footnote.

The verdict is quoted, never summarised

The record carries the gate's own last line. A summary is a second source of truth about whether something passed.

Room does

- Reads which suites ran and which did not.
- Names one path in their repository whose owner is currently ambiguous.

Ask

If your graph were wrong by one directory, which failure would you never see?

Cohort module five, two hours

Build Self-Healing Engineering Loops

Recovering from a failed check without a human restarting anything, and knowing when to give up. The second half is about stopping, which is the part people skip.

Ideas taught here: **two roles**, **how a loop ends**, **the time budget**.

0:00 to 0:35

Handing back a reason, not a rejection

Goal A failed check produces a next attempt that starts from what went wrong.

Say

- Show the useless version first: a failure returned as a status with no detail, and the retry that repeats the same mistake.
- Build the useful shape together. What failed, the output, the file, and what the task said should have been true.
- Be clear about why the roles are split. An agent grading its own output has already invested its reasoning in defending it.

Run

```
$ node steps/05-loop/demo.ts run 21
```

The loop is `steps/05-loop/loop.ts`.

Room does

- Watches a first attempt fail, and reads the packet that goes back.

- Watches the second attempt pass, having been told precisely what was wrong.

Ask *What does your pipeline hand back today when something fails, and would it help anyone?*

0:35 to 1:10

Three ways a loop ends

Goal Nobody leaves with a loop that can run indefinitely.

- Say**
- Ask what happens to their loop when the task is impossible. The honest answer is usually that nobody has tried.
 - Name the three endings and make them choose numbers for their own project rather than copying the lab's.
 - Reframe giving up. An item returned to a person with its attempts recorded is a result; a run still going is not.

Run

```
$ node steps/05-loop/demo.ts run 22 --impossible
```

- Room does**
- Sees attempts exhausted and the item escalated with its history intact.
 - Sets an attempt limit and a time limit in their own configuration.

Ask *What is the most you would spend on one item before a person should look at it?*

1:10 to 1:45

Timing each stage

Goal Everyone is timing each stage separately and can see which one to attack.

Say

- Ask them to guess the split before showing it. The guesses are almost always wrong in the same direction.
- Show the recorded numbers and let the surprise do the work. Then name the three usual culprits: repeated setup, repeated checks on finished work, and waiting on a person with nothing saying so.
- Give the actionable version. Reuse caches across workspaces, let the reviewer consume evidence rather than regenerate it, and make waiting visible.

Run

```
$ node steps/05-loop/demo.ts budget
```

Room does

- Reads a waterfall of stages for one completed item.
- Names the stage they would reduce first, and says how.

Ask *Which stage would you attack first on your own project, and what evidence do you have?*

1:45 to 2:00

Capstone scope

Goal Everyone has chosen the real item from their own backlog that they will demonstrate.

- Say**
- Give the selection rule: small enough to finish, real enough that somebody wanted it, and touching more than one area if possible.
 - Warn against the demonstration-shaped item. Choosing something invented for the occasion wastes the exercise.
-

Run Nothing. Backlogs open, choices written in the shared document.

Room does Commits to one item by name and posts it where the group can see.

Ask *Is anyone's choice too large? Say it now rather than on demonstration day.*

1:35 to 1:50

Retrying from a failed check

Goal They can distinguish a retry that uses the reason from one that merely repeats.

- Say**
- A loop that retries without reading the failure is a slower way to fail the same way.
 - Compare the two attempts and find the sentence in the second that only exists because of the first.
 - Cap the attempts out loud: twice with the reason attached, then a person.
-

Run

```
$ make score
```

1st Open Never report a negative remaining balance #604
Param-Harrison wants to merge 2 commits into `main` from `gate/remaining-clamped`

test must be revised or removed — its assertion no longer holds for overspend cases, and removing it requires justification.

2. Update the `assess()` `docstring` (lib.rs:98-99) to describe the clamped behavior instead of the negative-reporting behavior. Clarify why clamping is preferred over the previous explicitly-chosen design.
3. Update the module-level invariant documentation (lib.rs:5-6) to reflect that `spent + remaining == limit` holds only when `spent` does not exceed the limit.

Alternatively, if the existing design of negative remaining is intentional and correct (which the tests and docs assert), close this PR without merging.

Assumptions

- The author's intent is cosmetic — "reads badly in the console" — not a correctness fix. The existing `format_minor` function already handles negative values, so this is a design preference, not a bug.
- The PR is a first-time contribution from this author (0 merged PRs), which warrants closer scrutiny of the design rationale.
- The absence of an issue means this design change was not discussed or approved before implementation.

Open questions

- Should the console display logic be updated instead of the money path? For example, the console could format overspend as "€0.00 remaining (overspent by €230.00)" without changing the budget engine's invariant.

Review runtime: openrouter/deepseek/deepseek-v4-flash, reasoning setting: off.

Param-Harrison added the `status:changes-requested` label 6 minutes ago

Param-Harrison commented 5 minutes ago

Nothing has changed since my last review pass. The previous verdict (`request: changes`) still stands: 2 tests fail, the documented invariant is broken, and the design change lacks issue context. The three requested changes are still unaddressed.

Attempt two, beginning from the failure text rather than from the issue again.

Show on the board

Work to a card that retried to its session to attempt two

The second attempt starts from the reason

Find the sentence in attempt two that only exists because attempt one failed. If there is none, the loop is repeating rather than recovering, and it will repeat for ever.

An error can survive its own fix

A decision that succeeded on its second attempt still carries the text of the first failure. Anyone treating the presence of an error as failure will report healthy runs as broken.

Twice, then a person

The charter caps attempts and says what happens after. Ask each table for their number and what they would do with the third failure.

Room does

- Finds the failure text carried into the second attempt.
- Sets a retry limit for their own repository and writes the reason beside it.

Ask

How many times should something fail before the failure becomes yours rather than its?

Cohort module six, two hours

Orchestrate and Ship Verified PRs

Everything connects. The last half hour is the build against buy conversation, answered by mapping what they built onto a hosted product.

Ideas taught here: **durable state, the delivered request, who decides, the record, stages on a board, policy that stays yours.**

0:00 to 0:45

One workflow, six layers

Goal Each person's pieces run as a single command against a real item in their own repository.

Say

- Resist rebuilding anything. This session connects what exists, and the connecting code should be short and boring.
- Walk the seams rather than the parts: what triage hands to implementation, what implementation hands to checking, what checking hands to delivery.
- Point out that every handoff is written somewhere durable, so a run that dies can be picked up rather than restarted.

Run

```
$ make demo STEP=06
```

Room does

- Runs the whole sequence on their own item, unattended.

- Reads their own handoff records and confirms each stage could be resumed.

Ask

Kill your run halfway. Can the next one continue from what it left behind?

0:45 to 1:15

What a reviewer receives

Goal

Their requests arrive with enough attached that a colleague could judge one in a few minutes.

Say

- Have them review each other's output rather than their own. Reviewing your own factory tells you nothing.
- Collect what reviewers wanted and did not find, and add those to the template on the spot.
- Be firm about the boundary. Nothing merges itself at any tier, and the enforcement belongs in the repository settings rather than in a script.

Run

```
$ node steps/06-delivery/demo.ts open-pr  
$ node steps/06-delivery/demo.ts try-merge
```

Room does

- Reviews a peer's request and reports what was missing.
- Watches an attempted merge blocked, and checks their own protection settings.

Ask

What did your partner need that your template did not give them?

1:15 to 1:45

Buying instead of building

Goal Everyone can decide for their own organisation, with reasons that are not about preference.

Say

- Open the hosted product and map its stages onto their own, out loud, one at a time. The correspondence is close and that is the lesson.
- Show where their policy lands on the other substrate. It is a function that allows or refuses a move, holding the rules they wrote in week one.
- Frame the decision honestly. This is about who maintains the parts you now understand, not about whether to have them.

Run

```
$ make demo STEP=07
```

The mapping is written out in `steps/07-mastra/README.md`.

Room does

- Sees a board whose columns are the stages they built by hand.
- Sees their protected path expressed as a policy function on the other substrate.

Ask *Which layers would you hand to a vendor, and which would you never give up?*

1:45 to 2:00

Demonstration day

Goal Everyone knows the shape of what they will show and how long they have.

- Say**
- Give the running order and the timing, and say what you will ask each of them.
 - Set the expectation that a refusal or an escalation shows more than a clean pass. It shows the limits working.
-

Run Nothing. Schedule on screen.

Room does Picks a slot and confirms their item still runs end to end.

Ask *Anyone whose item stopped working this week? Bring it to Friday instead of debugging alone.*

1:35 to 1:50

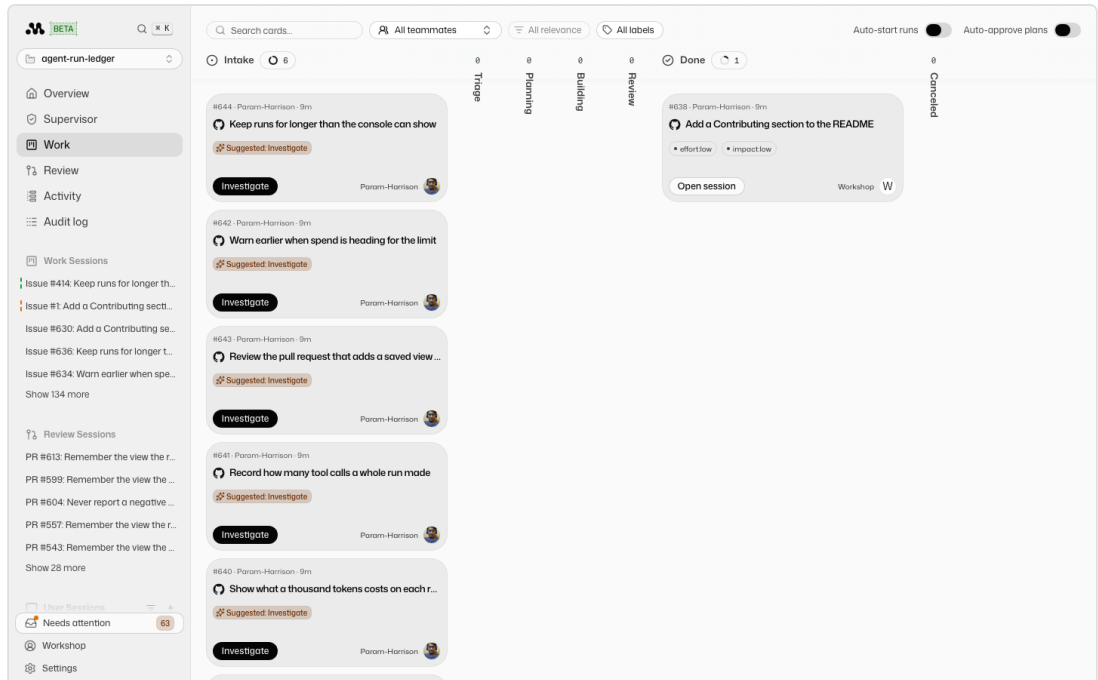
A finished board is not a shipped change

Goal They stop treating a board reaching done as a claim that anything shipped.

- Say**
- Done on the board means the machine has nothing left to do, which is not the same as merged.
 - Two boards, and this is the one for issues; the pull request lives on the other with its own phases.
 - Anyone who conflates them will report a week of work that is still sitting unreviewed.
-

Run

```
$ make e2e
```



The item reaches done while its pull request stays open.

Show on the board

Review to its own phases to the same change on Work to done, unmerged

Done means the machine is finished

The work item reaches done when its pull request exists. Nothing has merged, nobody has read it, and the board is not claiming otherwise.

The same change, two boards

Find it on Work as an issue and on Review as a pull request. They have different phases because they are different questions: is this worth doing, and is this done well.

A re-review needs a push

The review board has no review-to-review step. Fixing the branch is what starts the second pass, and asking the board to review again does nothing at all.

Room does

- Finds the same change represented on both boards.
- Writes the sentence they would use in a status update, and has it corrected.

Ask

Your dashboard says twelve done. How many of those can a customer use?

Leadership workshop, one day

Design an AI-Native Software Factory for Your Organization

A working session for people who decide how engineering works. They do not write code here. They leave with an operating model, an autonomy matrix and a sequenced roadmap.

Ideas taught here: **the six jobs, a written policy, grading autonomy, paths off limits, capacity, accountability, measuring, ownership.**

Evidence to bring: **what adoption measured, felt against measured speed, the named risk classes, routing as published practice.**

0:00 to 1:15

Writing down the rules you already follow

Goal Each team's current rules are on paper, and the variation between teams is visible to everyone in the room.

Say

- Open by asserting that they already have an operating model. It is informal, it differs by team, and nobody has compared the versions.
- Have each team write theirs on a card: what a machine may do unattended, what needs approval, and who grants exceptions.
- Put the cards side by side and let the spread speak. That gap is the reason for the day, and it is more persuasive than any slide.

Run

Nothing technical. Cards on a wall.

Room does

- Produces one card per team and reads two others aloud.
- Marks every place two teams contradict each other.

Ask

Where two of your teams disagree, which one is right, and who would decide that today?

1:30 to 3:00

The autonomy matrix

Goal

Their estate is sorted by consequence, with approval placed where the risk actually is.

Say

- Kill the difficulty axis early, using an example from their own estate rather than a generic one.
- Run the sort as a group exercise across their real services, and let the arguments happen. The arguments are the product.
- Push back whenever everything lands in the strictest column. A model where nothing may proceed is a model nobody will follow.

Run

```
$ node steps/01-boundary/demo.ts graph
```

Show the worked example once, as a shape to copy, then put it away.

Room does

- Places every significant service in a column, with a named owner.
- Agrees who may move something between columns and how that gets recorded.

Ask

Which service caused the longest argument? That is the one your written policy most needs to cover.

3:45 to 5:00

Protecting the standard you already have

Goal They know which checks must run on every change and which belong before release, and where their current setup fails open.

Say

- Ask what a machine could produce today that would pass their pipeline and still be wrong. Let the room be uncomfortable.
- Introduce failing closed as a governance property, not a technical one. A missing check is invisible in every report they receive.
- Explain the independent read in terms they own: an assurance step, with a cost per change they can decide to pay.

Run

```
$ bash steps/04-verification/gates.sh --without-tests
```

Room does

- Sees a missing check produce something that is neither a pass nor a failure.
- Lists the checks that will be required across every team.

Ask *How would you learn, this month, that one of your teams quietly disabled a required check?*

5:00 to 5:30

Who is accountable for a change nobody wrote

Goal Accountability is assigned explicitly, and the capacity limit is a number rather than a feeling.

- Say**
- Put the question bluntly, because it is the one their board will ask them. A machine wrote it, someone approved it, and something went wrong.
 - Settle it the same way as any other change: a named engineer approved it and owns it.
 - Then give them the constraint that follows. Approval capacity is finite, so the pipeline must stop producing when it is full.
-

Run

```
$ node steps/01-boundary/demo.ts capacity
```

- Room does**
- Writes the accountability rule in one sentence.
 - Sets a review capacity per team and says how it will be measured.
-

Ask *How many changes per week can each of your teams genuinely stand behind?*

5:30 to 6:00

Sequencing the rollout

Goal They leave with an ordered plan whose first step is small enough to happen next week.

- Say**
- Insist the first move is boring and cheap. Trust is built by a small thing that works, not by a programme.
 - Have them pick the first workflow by two criteria: it happens often, and getting it wrong is recoverable.
 - Name what they will measure before starting, so the second conversation is about numbers rather than impressions.
-

Run

Nothing. The roadmap template on screen.

Room does

- Fills in the first three steps with owners and dates.
 - Agrees the baseline they will compare against later.
-

Ask

What will you be able to show in six weeks, and who needs to see it?

2:20 to 2:40

Four organisational shapes

Goal

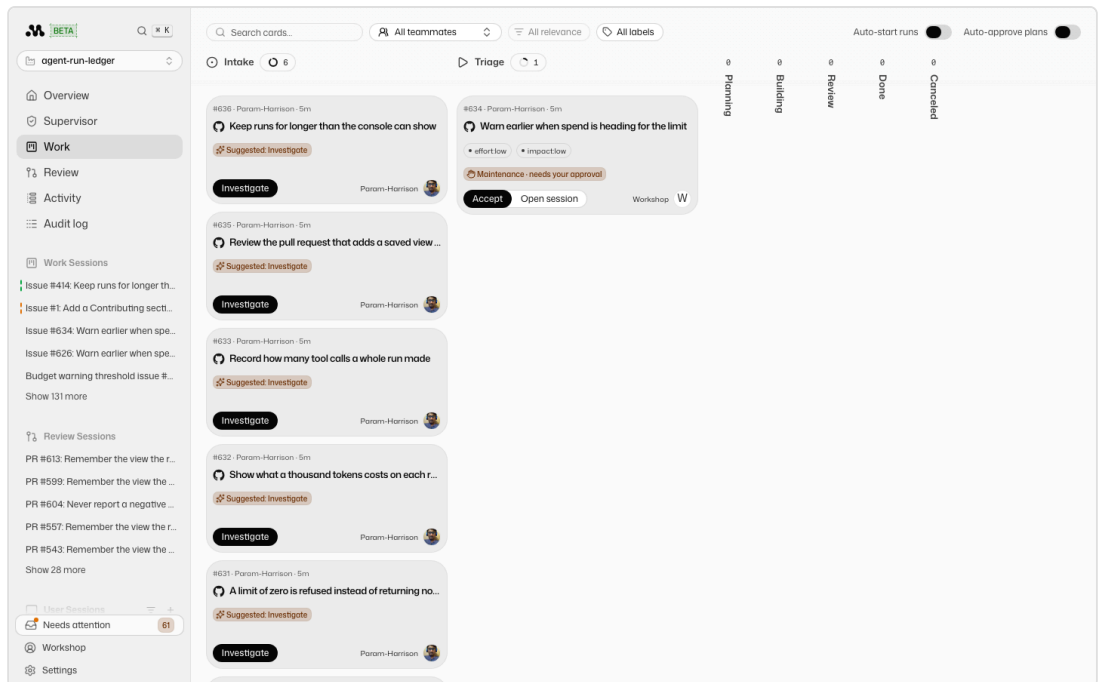
Each leader has chosen a shape for one real team and can defend the cost it imposes.

Say

- Switch the shape live and re-run the same item. The room then sees the rules doing the work, not the model.
 - None of the four is correct. Each is a position about who carries the cost of being wrong.
 - Push back on anyone choosing the strictest by default. It has a price, and people who are not in this room pay it in throughput.
-

Run

```
$ make profile NAME=solo  
$ make profile NAME=enterprise
```



The same item, under the strictest of the four shapes.

Show on the board

Settings to the two switches to Work to the money item

A shape is a commit, not a setting

make profile writes the charter and the ownership graph and pushes them, because a run clones from the remote. A shape that exists only on somebody's laptop is a shape no agent will ever read.

The queue cap is the number to argue about

Every shape declares how many decisions may wait on a person. That number, not the model, is what decides whether the factory helps or buries the team.

Nobody in the room pays the cost of the strictest shape

Pushing everything to refuse feels responsible and is paid for in throughput by people who are not at this table. Ask who they are and whether they agreed.

Room does

- Picks a shape for one named team and writes the trade-off it accepts.
- Names the first rule they would relax after a quarter of evidence.

Ask

Who in your organisation currently pays for a change being wrong, and do they know?

Weekly hour, plus the final session

Office hours and capstone demo day

Unscripted by design. This sheet exists so the hour has a shape when nobody brings anything, and so the last one runs to time.

Ideas taught here: **records, endings.**

First ten minutes

Collect before solving

Goal Every blocker in the room is written down before any of them is worked on.

Say

- Ask for problems in one line each and write them all up before touching any of them.
- Look for the repeated one. When three people are stuck the same way, that is a teaching gap rather than three bugs.
- Say the order out loud so nobody is waiting without knowing whether their turn is coming.

Run Nothing. A list on screen.

Room does Contributes one line each, including those who came only to listen.

Ask *Anything here that more than one of you recognises?*

As they come up

The four recurring blockers

Goal Recurring problems get a fast answer so the hour goes to the interesting ones.

Say

- A new workspace with no dependencies installed. Answer with caching and a preparation command rather than a longer setup step.
- Context that never arrives. Almost always a path the ownership graph does not claim, so run the graph check first.
- A check that passes when it should not. Apply the reverting proof before debugging anything else.
- A run that never ends. Look for the missing limit rather than the slow model.

Run

```
$ node steps/02-execution/demo.ts selftest
$ node steps/01-boundary/demo.ts graph
```

Room does Runs the same two commands on their own project before describing the symptom.

Ask *What did the self-check say? Read it out before we theorise.*

Final session only

Demonstrations

Goal Everyone shows their own item moving, and gets one useful piece of feedback from the group.

Say

- Give each person a fixed slot and hold it, so the last presenter gets the same time as the first.
- Ask the same two questions of everyone: which layer took the longest to get right, and what did it refuse to do.
- Direct the feedback towards what the group can use rather than towards praise.

Run

Their machines, their repositories. Yours stays closed.

Room does

- Shows an item becoming a reviewed request on their own codebase.
- Names one thing they would build differently.

Ask

What will you take back to your team on Monday, in one sentence?

0:20 to 0:40

Reading a run you did not perform

Goal

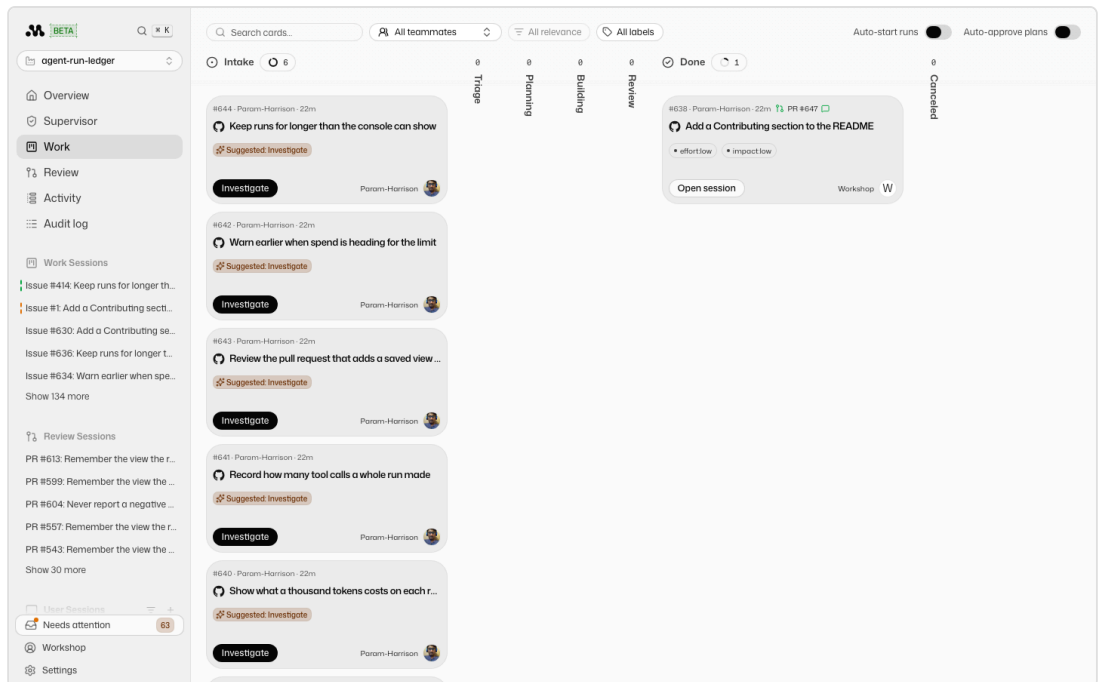
Everyone has read one run they did not perform and formed a view without being told the story.

Say

- Bring up a capstone board and review it cold, with no narration from its author.
- Judge the evidence rather than the outcome, because a green run with no proof is the thing this whole course argues against.
- Let the author respond only after the room has committed to a verdict.

Run

```
$ make status  
$ make score
```



Pull requests on their own board, with their own phases.

Show on the board

Review to somebody else's board to a pull request to the verdict

Judge the evidence, not the outcome

A green run with no proof is the thing this course argues against. Ask what would have to be true for this verdict to be wrong, before the author says anything.

The author speaks last

Let the room commit to a view first. A narrated demo teaches the narrator, and everyone else watches somebody else be competent.

One number each, and what it cost

Every capstone reports a coverage figure and a score. Ask which of the two they trust and why, because the gap between them is the only interesting part.

Room does

- States a verdict on a run they did not make.
- Names the one piece of evidence that would have changed it.

Ask

What did you believe about this run before the author spoke, and what changed after?